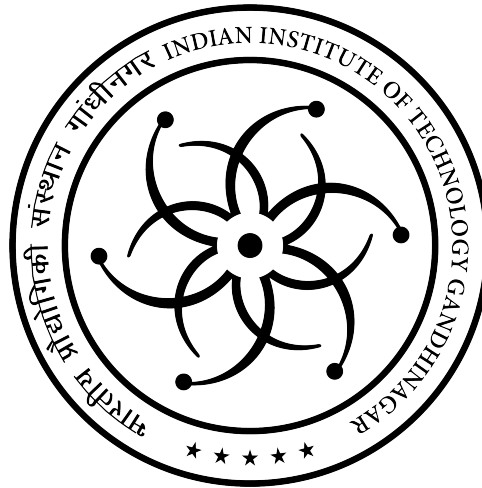


Lab Assignment - 11

3 November 2025



CS 202

Software Tools and Techniques for CSE

INDIAN INSTITUTE OF TECHNOLOGY GANDHINAGAR

Palaj, Gandhinagar - 382355

Events and Delegates in C# Windows Forms Applications

Submitted by

Gaurav Budhwani

22110085

Abstract

This report details the work carried out for Lab 11, focusing on event-driven programming in C#. The lab provided a deep understanding of the publisher-subscriber model through the implementation of events, delegates, and custom event arguments in a Windows Forms (WinForms) application.

A C# WinForms application named **EventPlayground** was developed to demonstrate these concepts. The application features interactive controls that communicate using custom-defined events rather than default control events. Key implementations include defining custom **delegate** types, declaring **event** members, creating a custom **EventArgs** class (**ColorEventArgs**) to pass state data, and subscribing multiple handler methods to a single event (multicast).

The report also contains the analysis of six output-reasoning problems. These exercises exposed subtle but critical behaviors of C#, including delegate invocation order, handling of **ref** parameters in delegates, re-entrant event calls, and the consequences of nested event invocation.

Contents

1	Introduction, Setup, and Tools	3
1.1	Objectives	3
1.2	Environment and Tools	3
1.3	One-time Setup Steps	3
2	Methodology and Execution	6
2.1	GUI Design	6
2.2	Custom Event Arguments (<code>ColorEventArgs.cs</code>)	7
2.3	Form Logic (<code>Form1.cs</code>)	8
3	Results and Analysis	10
3.1	Application Execution	10
3.2	Key Insights	12
4	Output Reasoning	13
4.0.1	Output Reasoning (Level 0) Q1	13
4.0.2	Output Reasoning (Level 0) Q2	14
4.0.3	Output Reasoning (Level 1) Q1	15
4.0.4	Output Reasoning (Level 1) Q2	16
4.0.5	Output Reasoning (Level 2) Q1	17
4.0.6	Output Reasoning (Level 2) Q2	18
5	Discussion and Conclusion	19
5.1	Discussion	19
5.2	Limitations and Potential Improvements	20
5.3	Conclusion	20
6	References	21

1 Introduction, Setup, and Tools

1.1 Objectives

The primary objectives of this lab assignment were to deepen the understanding of event handling and delegates in C#. By the end of this lab, I was able to:

- Implement and handle custom events in C# Windows Forms applications using the publisher-subscriber model.
- Design and manage interactive form controls that respond dynamically to user actions through event-driven logic.
- Apply multicast event handling to connect multiple subscribers (methods) to a single event.
- Use custom **EventArgs** classes to pass contextual information between event publishers and subscribers.

1.2 Environment and Tools

The lab was developed and executed using the following environment:

- **Operating System:** Windows 11
- **IDE:** Visual Studio 2022 (Community Edition)
- **Target Framework:** .NET 8.0 (compatible with the .NET SDK requirement)
- **Language:** C# (latest stable version)

1.3 One-time Setup Steps

The setup process involved creating a new Windows Forms project in Visual Studio, which provides the necessary template and GUI designer.

1. Launched Visual Studio 2022 and selected "Create a new project".
2. Searched for and selected the **Windows Forms App** template for C# and .NET (Figure 1).
3. Configured the project, naming it **EventPlayground** and setting the save location (Figure 2).
4. The IDE loaded the default **Form1.cs** in the designer view (Figure 3).
5. A new class file was added to the project (**Project > Add Class...**) and named **ColorEventArgs.cs** to hold custom event data (Figure 4).

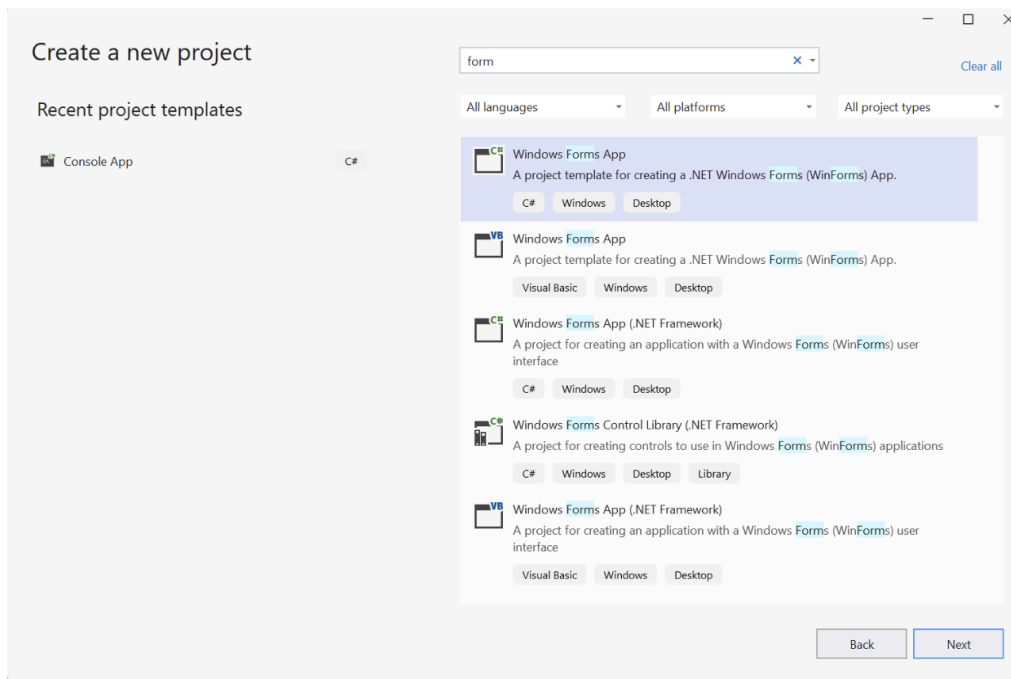


Figure 1: Selecting the "Windows Forms App" template

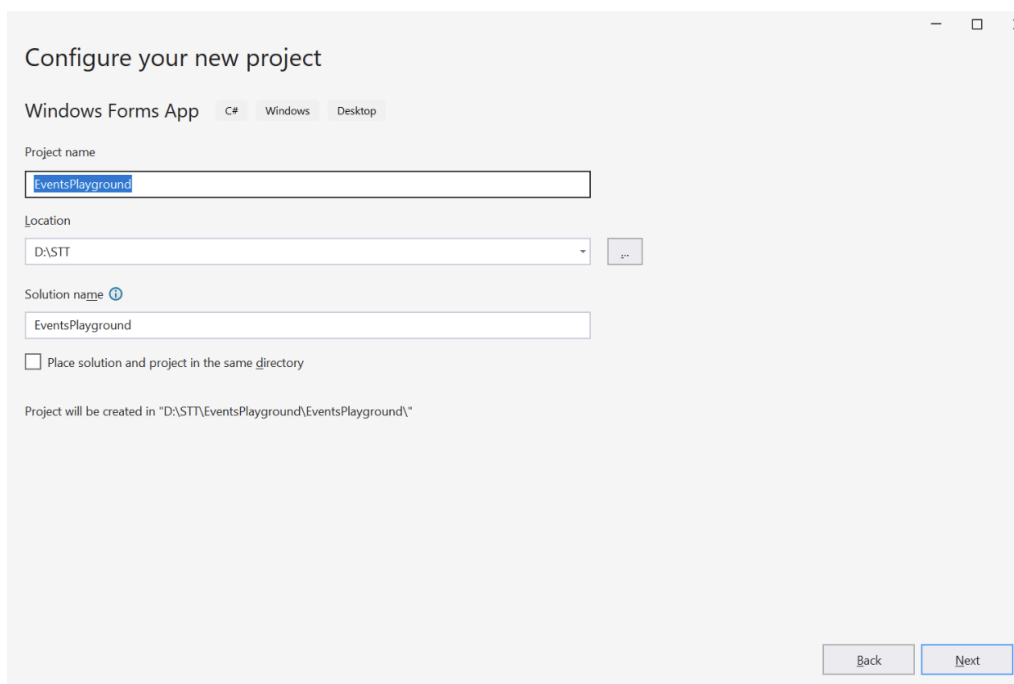


Figure 2: Configuring the project name and location.

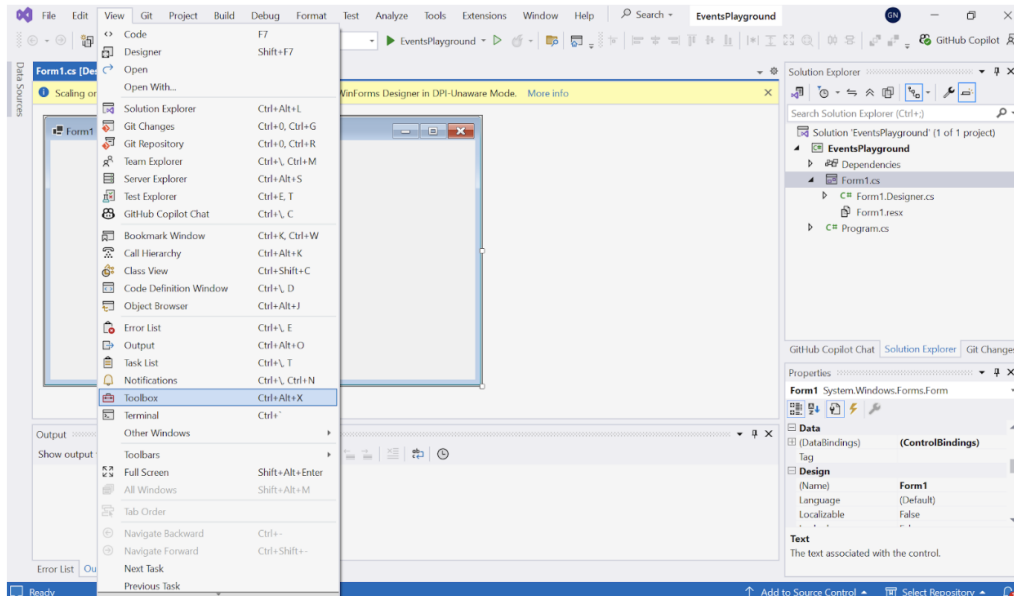


Figure 3: The Visual Studio IDE, Solution Explorer, and Properties window.

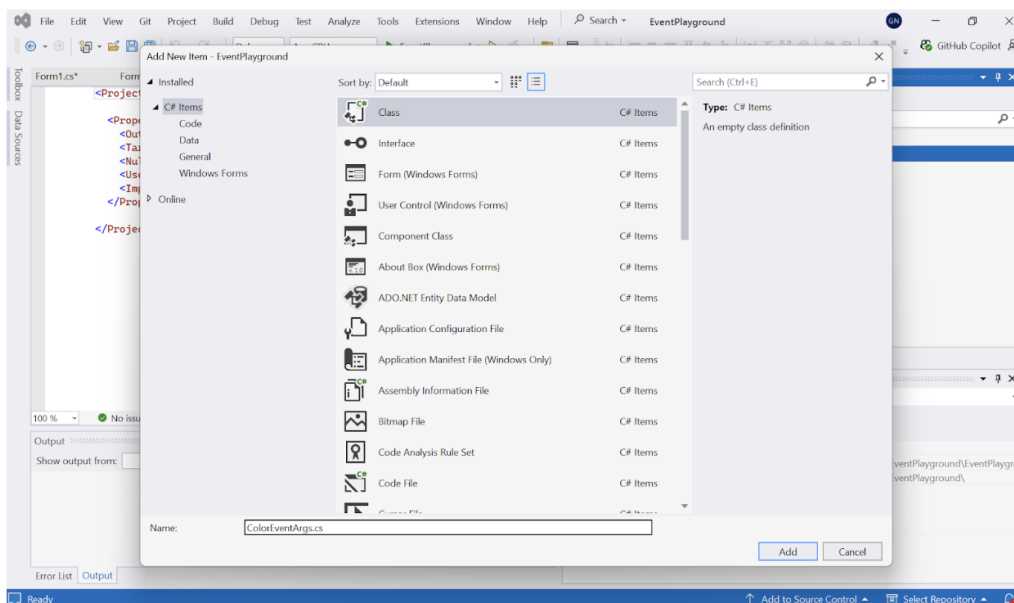


Figure 4: Adding the new ColorEventArgs.cs class file.

2 Methodology and Execution

The lab was executed in two main parts: first, building the `EventPlayground` GUI application, and second, analyzing the output reasoning code snippets.

2.1 GUI Design

The form was designed using the Visual Studio WinForms designer. The following controls were added and configured via the Properties window:

- **Label:** Named `lblDisplay`, with its `Text` property set to "Welcome to Events Lab" (Figure 5).
- **ComboBox:** Named `cmbColors`. The items "Red", "Green", and "Blue" were added to its `Items` collection (Figure 6). Its `DropDownStyle` was set to `DropDownList`.
- **Button:** Named `btnChangeColor`, with `Text` set to "Change Color" (Figure 7).
- **Button:** Named `btnChangeText`, with `Text` set to "Change Text" (Figure 8).

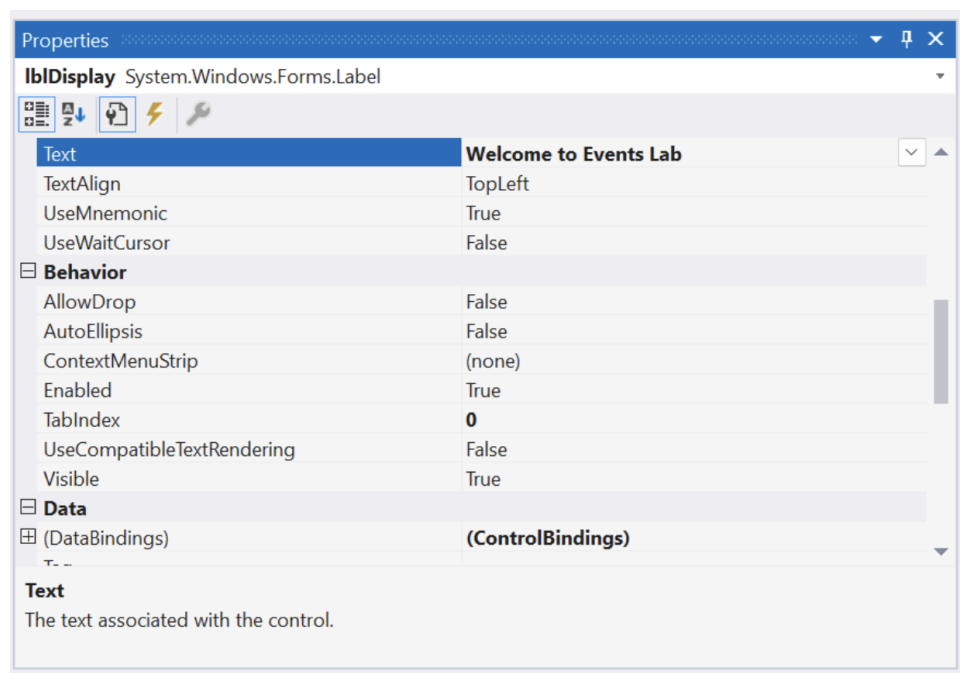


Figure 5: Properties for `lblDisplay`.

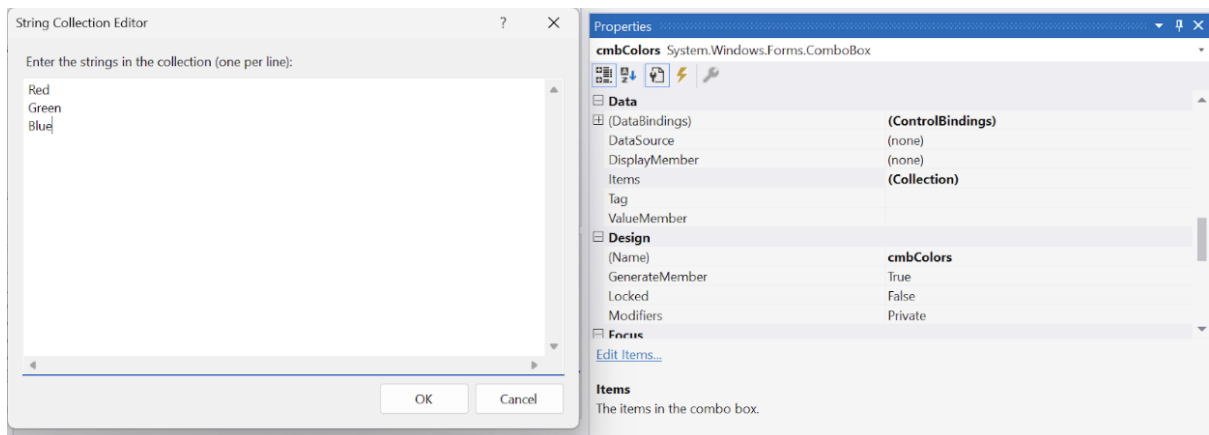


Figure 6: Adding items to cmbColors.

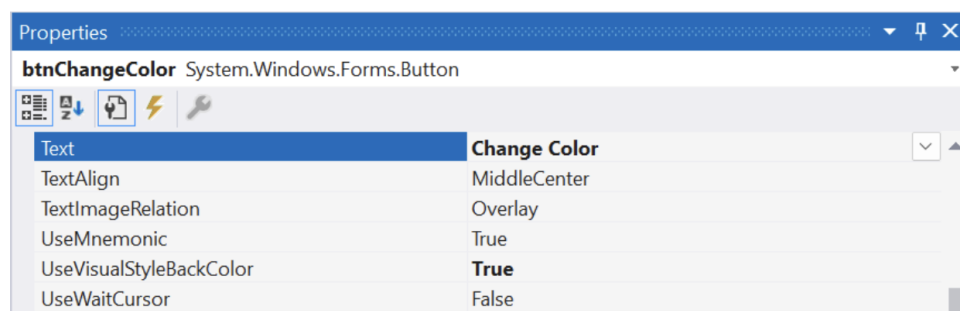


Figure 7: Properties for btnChangeColor.

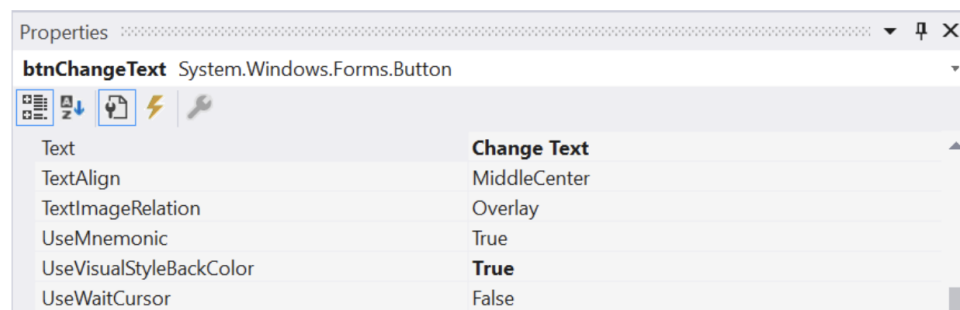


Figure 8: Properties for btnChangeText (Figure 8 from stt lab 11.pdf).

2.2 Custom Event Arguments (ColorEventArgs.cs)

To pass the selected color name during the event, a custom class inheriting from EventArgs was created.

```

1 using System;
2
3 namespace EventPlayground
4 {
5     public class ColorEventArgs : EventArgs
6     {
7         public string ColorName { get; }
8
9         public ColorEventArgs(string colorName)
10        {

```

```

11         ColorName = colorName;
12     }
13 }
14 }

```

Listing 1: ColorEventArgs.cs: Custom class for event data.

2.3 Form Logic (Form1.cs)

The main form logic implements the publisher-subscriber pattern.

1. **Delegates and Events:** Custom delegates (ColorChangedHandler, TextChangedHandler) and their corresponding public events (ColorChangedEvent, TextChangedEvent) were declared.
2. **Constructor:** In the Form1 constructor, handler methods (UpdateLabelColor, ShowNotification, UpdateLabelText) were subscribed to their respective events.
3. **Publishers:** The button click handlers (btnChangeColor_Click, button1_Click) act as publishers. They gather data, create the EventArgs (if needed), and invoke the event using Event?.Invoke(...).
4. **Subscribers:** The private methods UpdateLabelColor, ShowNotification, and UpdateLabelText act as subscribers, executing their logic when the event is fired.

The code for Form1.cs is shown in Listing 2.

```

1 using System;
2 using System.Drawing;
3 using System.Windows.Forms;
4
5 namespace EventPlayground
6 {
7     public partial class Form1 : Form
8     {
9         // Delegates
10         public delegate void ColorChangedHandler(object sender, ColorEventArgs
11         e);
12         public delegate void TextChangedHandler(object sender, string newText);
13
14         // Events
15         public event ColorChangedHandler ColorChangedEvent;
16         public event TextChangedHandler TextChangedEvent;
17
18         public Form1()
19         {
20             InitializeComponent();
21
22             // Subscribe multiple handlers to ColorChangedEvent (Multicast)
23             ColorChangedEvent += UpdateLabelColor;
24             ColorChangedEvent += ShowNotification;
25
26             // Subscribe single handler to TextChangedEvent
27             TextChangedEvent += UpdateLabelText;
28
29             // Manual hook for changing color
30             // Note: The designer may add this automatically
31             btnChangeColor.Click += btnChangeColor_Click;
32
33             // Note: The handler 'button1_Click' corresponds to 'btnChangeText'
34             btnChangeText.Click += button1_Click;
35         }
36     }
37 }

```

```

35
36     private void label1_Click(object sender, EventArgs e)
37     {
38     }
39
40     // Publisher for TextChangedEvent
41     private void button1_Click(object sender, EventArgs e)
42     {
43         // Fire TextChangedEvent
44         string newText = DateTime.Now.ToString("dd-MM-yyyy HH:mm:ss");
45         TextChangedEvent?.Invoke(this, newText);
46     }
47
48     // Publisher for ColorChangedEvent
49     private void btnChangeColor_Click(object sender, EventArgs e)
50     {
51         string selected = cmbColors.SelectedItem?.ToString();
52
53         if (string.IsNullOrEmpty(selected))
54             return;
55
56         // Fire ColorChangedEvent
57         ColorChangedEvent?.Invoke(this, new ColorEventArgs(selected));
58     }
59
60     // Subscriber 1 for ColorChangedEvent
61     private void UpdateLabelColor(object sender, ColorEventArgs e)
62     {
63         switch (e.ColorName)
64         {
65             case "Red":
66                 lblDisplay.ForeColor = Color.Red;
67                 break;
68             case "Green":
69                 lblDisplay.ForeColor = Color.Green;
70                 break;
71             case "Blue":
72                 lblDisplay.ForeColor = Color.Blue;
73                 break;
74         }
75     }
76
77     // Subscriber 2 for ColorChangedEvent
78     private void ShowNotification(object sender, ColorEventArgs e)
79     {
80         MessageBox.Show($"Color changed to: {e.ColorName}");
81     }
82
83     // Subscriber for TextChangedEvent
84     private void UpdateLabelText(object sender, string newText)
85     {
86         lblDisplay.Text = newText;
87     }
88 }
89 }

```

Listing 2: Form1.cs: Implementing the event-driven logic.

3 Results and Analysis

This section presents the observed outputs from the `EventPlayground` application, confirming the correct implementation of the event-driven design.

3.1 Application Execution

The application was built and run.

- **Initial State:** The application launches with the label displaying "Welcome to Events Lab" and the combo box empty (Figure 9).
- **Color Change Event:** A color (e.g., "Red") was selected from `cmbColors` and `btnChangeColor` was clicked.
 - This single click successfully triggered **both** subscribed methods.
 - The label's foreground color changed to red.
 - A `MessageBox` appeared with the text "Color changed to: Red" (Figure 10).
- This demonstrates a successful **multicast event**. The same behavior was observed for "Green" (Figure 11) and "Blue" (Figure 12).
- **Text Change Event:** The `btnChangeText` was clicked.
 - This triggered the `TextChangedEvent`.
 - The sole subscriber, `UpdateLabelText`, executed, changing the `lblDisplay.Text` property to the current date and time (Figure 13).

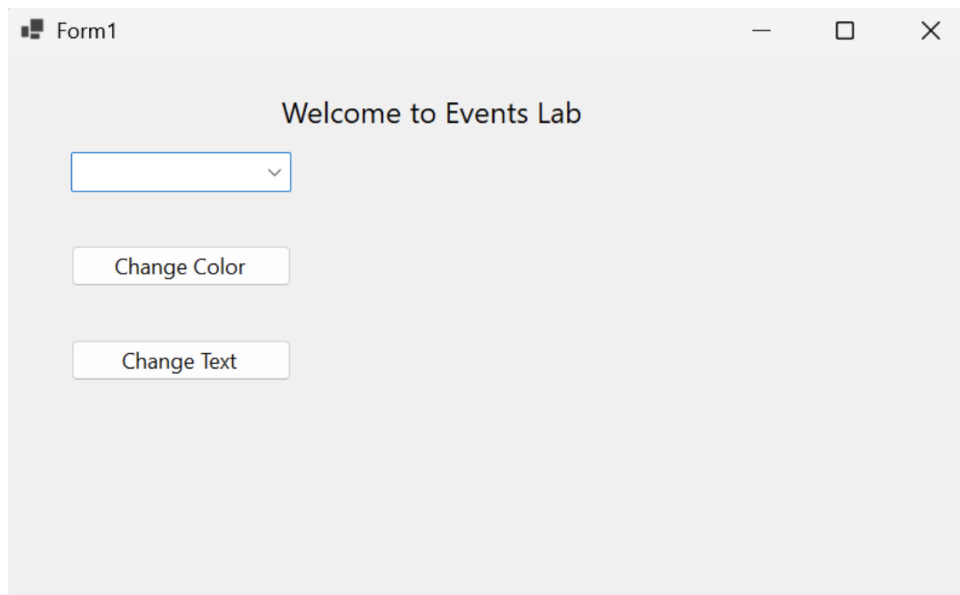


Figure 9: Initial application state (Figure 9 from `stt_lab_11.pdf`).

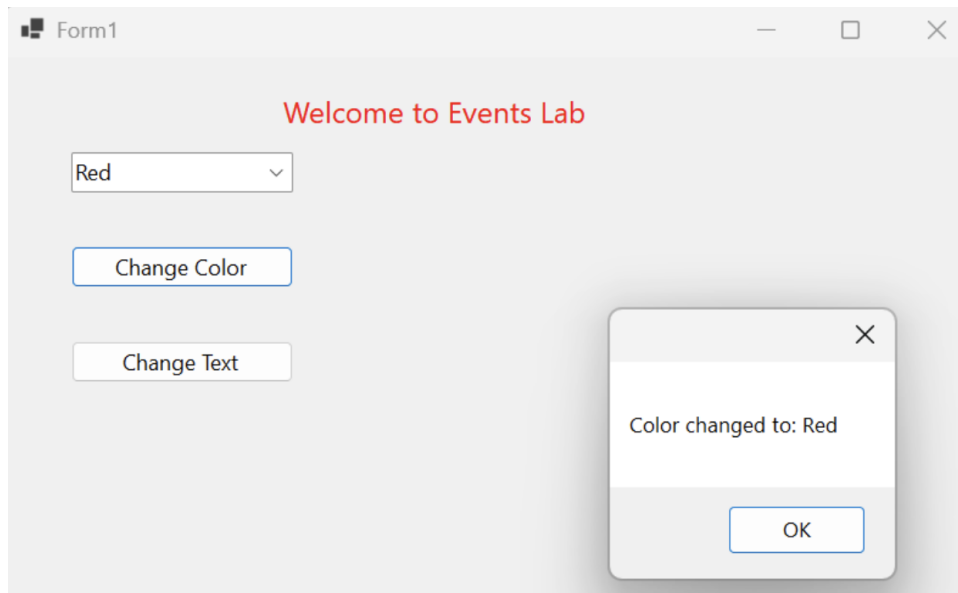


Figure 10: ColorChangedEvent (Red) firing both label and message box handlers (Figure 10).

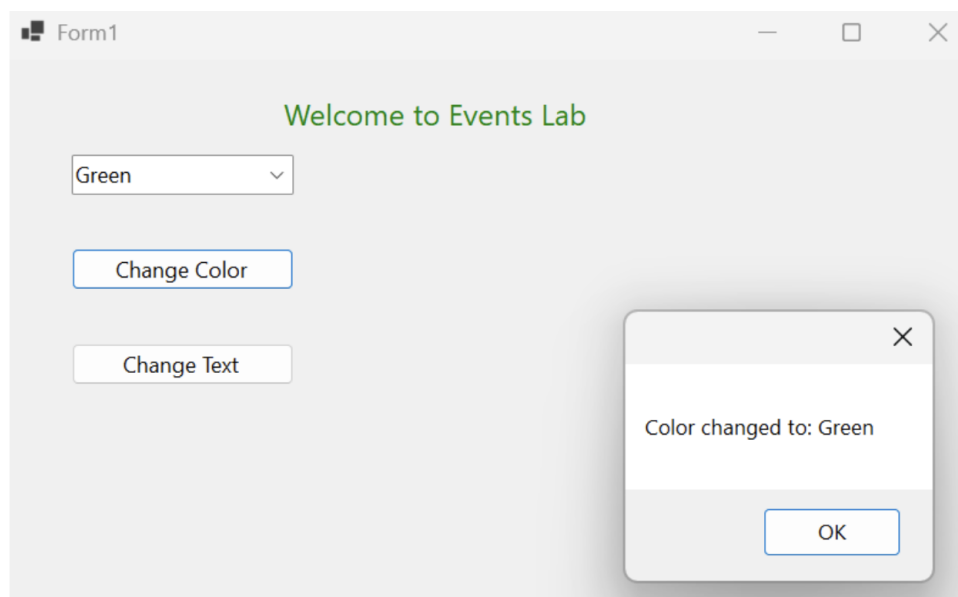


Figure 11: ColorChangedEvent (Green) firing both handlers (Figure 11).

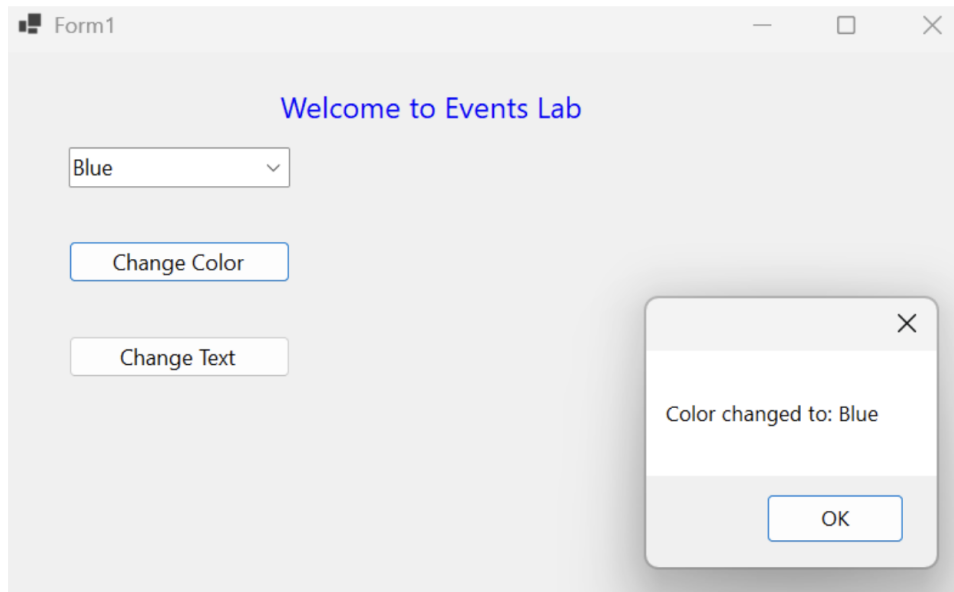


Figure 12: ColorChangedEvent (Blue) firing both handlers (Figure 12).

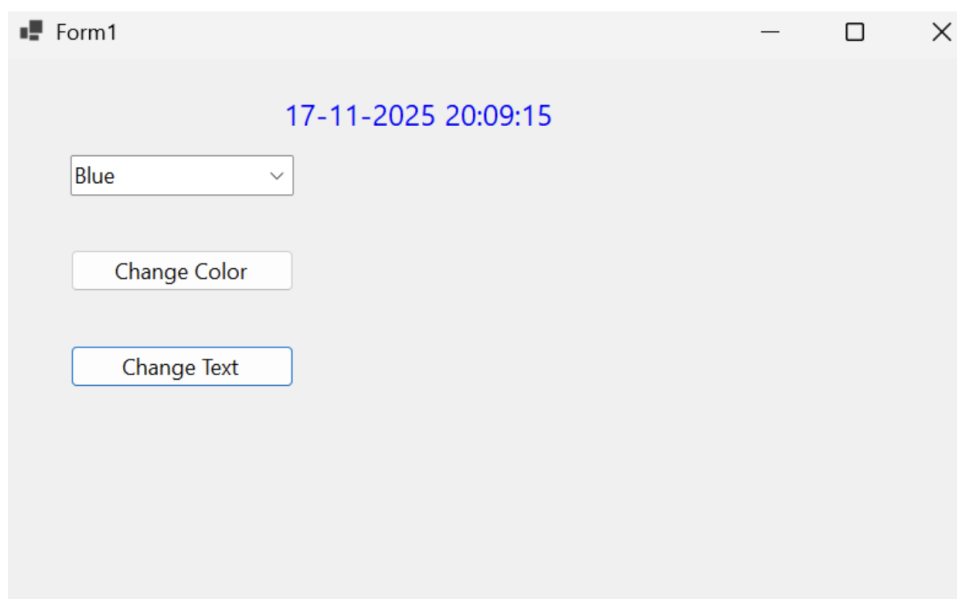


Figure 13: TextChangedEvent firing and updating the label text (Figure 13).

3.2 Key Insights

- **Decoupling:** The publisher-subscriber model is highly effective. The `btnChangeColor_Click` method (publisher) has no knowledge of what methods will execute. It only knows to *announce* the event. This decouples the button's logic from the label's logic and the notification logic.
- **Flexibility:** Because of this decoupling, we could add a third, fourth, or fifth subscriber (e.g., "LogColorChangeToFile") to the `ColorChangedEvent` without ever modifying the `btnChangeColor_Click` method.
- **Custom Data:** Using a custom `ColorEventArgs` class is far cleaner than passing raw strings. It provides a strongly-typed, clear contract for what data the event provides.

- **Multicasting:** Delegates' built-in support for multicasting (using +=) is a powerful feature that simplifies the implementation of complex notification systems, as seen with the color change event.
- **Event Safety:** Using the `event` keyword encapsulates the delegate, preventing external classes from clearing the subscriber list (e.g., `form.ColorChangedEvent = null;`) or invoking the event directly. They can only use += and -=.

4 Output Reasoning

This section analyzes the provided C# code snippets from the lab handout, explaining their execution flow and final output.

4.0.1 Output Reasoning (Level 0) Q1

```

1 delegate int Calc(int x, int y);
2 class Program
3 {
4     static int Add(int a, int b) { Console.Write("A"); return a + b; }
5     static int Mul(int a, int b) { Console.Write("M"); return a * b; }
6     static int Sub(int a, int b) { Console.Write("S"); return a - b; }
7     static void Main()
8     {
9         Calc c = Add;
10        c += Mul;
11        c += Sub;
12        c -= Add;
13        int res = c(2, 3);
14        Console.WriteLine(":" + res);
15    }
16 }

```

Listing 3: Code for L0 Q1 (Delegate Invocation Order and Return Value).

Expected Output:

MS:-1

Actual Output:

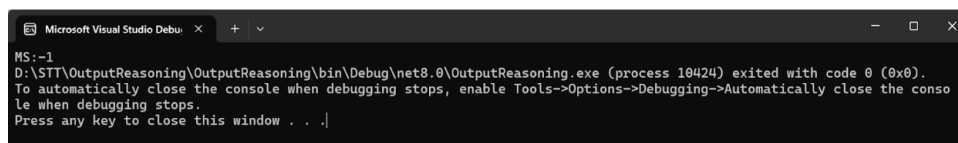


Figure 14: Actual Output for Level 0 Code 1.

Explanation (why):

1. A multicast delegate `c` is created.
2. Methods are added to its invocation list: `[Add]`, then `[Add, Mul]`, then `[Add, Mul, Sub]`.
3. `c -= Add;` removes the `Add` method. The final invocation list is `[Mul, Sub]`.
4. `c(2, 3)` is called. The methods execute in order.
5. `Mul(2, 3)` runs, prints "M", and returns 6.

6. `Sub(2, 3)` runs, prints "S", and returns -1.
7. For multicast delegates with a non-void return type, the variable `res` receives the return value of the **last** method executed.
8. Therefore, `res` is set to -1. The final print statement outputs `:-1`.

4.0.2 Output Reasoning (Level 0) Q2

```

1 delegate void ActionHandler(ref int x);
2 class Program
3 {
4     // Assuming typo 'a + 2;' is meant to be 'a += 2;'
5     static void Inc(ref int a) { a += 2; Console.Write("I" + a + " "); }
6     static void Dec(ref int a) { a--; Console.Write("D" + a + " "); }
7     static void Main()
8     {
9         int val = 3;
10        ActionHandler act = Inc;
11        act += Dec;
12        act(ref val);
13        Console.Write("F" + val);
14    }
15 }

```

Listing 4: Code for L0 Q2 (Delegates with 'ref' Parameters).

Expected Output:

I5 D4 F4

Actual Output:

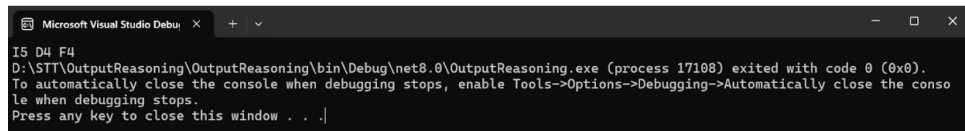


Figure 15: Actual Output for Level 0 Code 2.

Explanation (why):

1. `val` starts at 3. The delegate `act` has the list `[Inc, Dec]`.
2. `act(ref val)` is called. Because `val` is passed by `ref`, each method modifies the *original* `val` variable.
3. `Inc(ref val)` runs. `val` becomes $3 + 2 = 5$. It prints "I 5 ".
4. `Dec(ref val)` runs next. `val` becomes $5 - 1 = 4$. It prints "D 4 ".
5. The delegate call finishes. The variable `val` in `Main` is now 4.
6. The final statement prints "F4".

4.0.3 Output Reasoning (Level 1) Q1

```
1 class LimitEventArgs : EventArgs { /* ... */ }
2 class Counter
3 {
4     public event EventHandler<LimitEventArgs> LimitReached;
5     public event EventHandler<LimitEventArgs> MilestoneReached;
6     private int value = 0;
7     public void Increment()
8     {
9         value++;
10        Console.WriteLine(">" + value);
11        if (value % 2 == 0)
12            MilestoneReached?.Invoke(this, new LimitEventArgs(value));
13        if (value % 3 == 0)
14            LimitReached?.Invoke(this, new LimitEventArgs(value));
15    }
16 }
17 class Program
18 {
19     static void Main()
20     {
21         Counter c = new Counter();
22         c.LimitReached += (s, e) => Console.WriteLine("[L] " + e.CurrentValue + " ");
23         c.LimitReached += (s, e) => Console.WriteLine(" (Reset)");
24         c.MilestoneReached += (s, e) => {
25             Console.WriteLine("[M] " + e.CurrentValue + " ");
26             if (e.CurrentValue == 4)
27                 Console.WriteLine("{Alert}");
28         };
29         for (int i = 0; i < 6; i++)
30             c.Increment();
31     }
32 }
```

Listing 5: Code for L1 Q1 (Multiple Events and Subscribers).

Expected Output:

>1>2[M2]>3[L3] (Reset)>4[M4]{Alert}>5>6[M6] [L6] (Reset)

Actual Output:

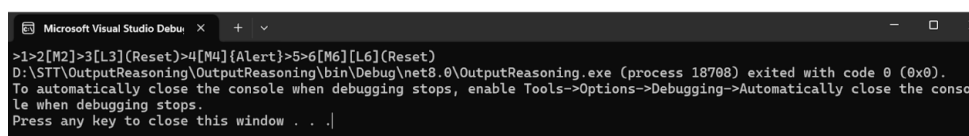


Figure 16: Actual Output for Level 1 Code 1.

Explanation (why): The `Increment()` method is called 6 times.

- **i=0 (value=1):** Prints >1.
- **i=1 (value=2):** Prints >2. `value % 2 == 0`. `MilestoneReached` fires. Prints [M2].
- **i=2 (value=3):** Prints >3. `value % 3 == 0`. `LimitReached` (multicast) fires. Prints [L3] then (Reset).
- **i=3 (value=4):** Prints >4. `value % 2 == 0`. `MilestoneReached` fires. Prints [M4]. The if condition is true. Prints {Alert}.

- **i=4 (value=5):** Prints >5.
- **i=5 (value=6):** Prints >6. `value % 2 == 0`. `MilestoneReached` fires. Prints [M6].
`value % 3 == 0`. `LimitReached` fires. Prints [L6] then (Reset).

4.0.4 Output Reasoning (Level 1) Q2

```

1 class TemperatureEventArgs : EventArgs { /* ... */ }
2 class TemperatureSensor
3 {
4     public event EventHandler<TemperatureEventArgs> TemperatureChanged;
5     private int temperature = 25;
6     public void UpdateTemperature(int newTemp)
7     {
8         int oldTemp = temperature;
9         temperature = newTemp;
10        // Assuming typo 'newTemp oldTemp' is 'newTemp - oldTemp'
11        if (Math.Abs(newTemp - oldTemp) > 5)
12            TemperatureChanged?.Invoke(this, new TemperatureEventArgs(oldTemp,
13            newTemp));
14    }
15 }
16 class Program
17 {
18     static void Main()
19     {
20         TemperatureSensor sensor = new TemperatureSensor();
21         sensor.TemperatureChanged += (s, e) =>
22             Console.WriteLine($"Temperature changed from {e.OldTemperature}C to
23             {e.NewTemperature}C");
24         sensor.TemperatureChanged += (s, e) => {
25             if (Math.Abs(e.NewTemperature - e.OldTemperature) > 10)
26                 Console.WriteLine(" Warning: Sudden change detected!");
27         };
28         sensor.UpdateTemperature(28);
29         sensor.UpdateTemperature(30);
30         sensor.UpdateTemperature(46);
31         sensor.UpdateTemperature(52);
32     }
33 }

```

Listing 6: Code for L1 Q2 (Conditional Event Firing).

Expected Output:

Temperature changed from 30 °C to 46 °C
Warning: Sudden change detected!
Temperature changed from 46 °C to 52 °C

Actual Output:

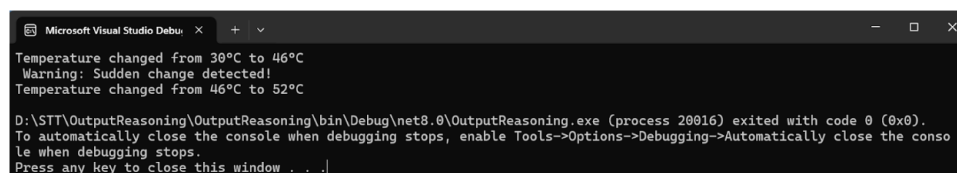


Figure 17: Actual Output for Level 1 Code 2.

Explanation (why): The event only fires if the change in temperature is > 5 . Initial temp is 25.

1. UpdateTemperature(28): old=25, new=28. Change is 3. 3 > 5 is false. No event.
2. UpdateTemperature(30): old=28, new=30. Change is 2. 2 > 5 is false. No event.
3. UpdateTemperature(46): old=30, new=46. Change is 16. 16 > 5 is true. Event fires.
 - Subscriber 1 prints "Temperature changed from 30C to 46C".
 - Subscriber 2 checks if 16 > 10. This is true. Prints "Warning: Sudden change detected!".
4. UpdateTemperature(52): old=46, new=52. Change is 6. 6 > 5 is true. Event fires.
 - Subscriber 1 prints "Temperature changed from 46C to 52C".
 - Subscriber 2 checks if 6 > 10. This is false. Prints nothing.

4.0.5 Output Reasoning (Level 2) Q1

```

1 class Notifier
2 {
3     public event EventHandler<NotifyEventArgs> OnNotify;
4     public void Trigger(string msg)
5     {
6         Console.WriteLine("[Start]");
7         OnNotify?.Invoke(this, new NotifyEventArgs(msg));
8         Console.WriteLine("[End]");
9     }
10 }
11 class Program
12 {
13     static void Main()
14     {
15         Notifier n = new Notifier();
16         n.OnNotify += (s, e) => {
17             Console.WriteLine("{0} {1} {2}", s, e.Message, "Pong");
18         };
19         n.OnNotify += (s, e) => {
20             Console.WriteLine("(Nested)");
21             if (e.Message == "Ping")
22                 ((Notifier)s).Trigger("Pong");
23         };
24         n.Trigger("Ping");
25     }
26 }

```

Listing 7: Code for L2 Q1 (Re-entrant / Nested Event Call).

Expected Output:

[Start] {Ping} (Nested) [Start] {Pong} (Nested) [End] [End]

Actual Output:

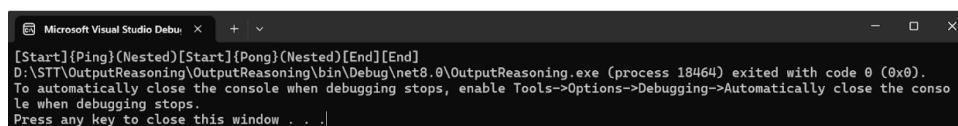


Figure 18: Actual Output for Level 2 Code 1.

Explanation (why): This demonstrates a re-entrant event call, where an event handler triggers the same event (or another) before the first call has finished.

1. `n.Trigger("Ping")` is called.
2. `Trigger("Ping")` prints `[Start]`.
3. `OnNotify` fires with `"Ping"`.
4. Subscriber 1 runs, printing `{Ping}`.
5. Subscriber 2 runs, printing `(Nested)`.
6. Inside Subscriber 2, `e.Message == "Ping"` is true.
7. `n.Trigger("Pong")` is called *before* `Trigger("Ping")` has finished.
 - `Trigger("Pong")` prints `[Start]`.
 - `OnNotify` fires with `"Pong"`.
 - Subscriber 1 runs, printing `{Pong}`.
 - Subscriber 2 runs, printing `(Nested)`.
 - Inside Subscriber 2, `e.Message == "Pong"` is false.
 - `Trigger("Pong")` finishes, printing `[End]`.
8. Control returns to `Trigger("Ping")`'s Subscriber 2, which finishes.
9. `Trigger("Ping")` finishes, printing `[End]`.

4.0.6 Output Reasoning (Level 2) Q2

```

1 class Sensor
2 {
3     public event EventHandler<AlertEventArgs> ThresholdReached;
4     public void Check(int value)
5     {
6         Console.WriteLine("[Check]");
7         // Assuming the Invoke is inside the 'if' block as intended
8         if (value > 50)
9         {
10             ThresholdReached?.Invoke(this, new AlertEventArgs("High"));
11         }
12         Console.WriteLine("[Done]");
13     }
14 }
15 class Program
16 {
17     static void Main()
18     {
19         Sensor s = new Sensor();
20         s.ThresholdReached += (sender, e) => {
21             Console.WriteLine("{ " + e.Info + " }");
22             if (e.Info == "High")
23                 ((Sensor)sender).Check(30);
24         };
25         s.ThresholdReached += (sender, e) => Console.WriteLine("(Alert)");
26         s.Check(80);
27     }
28 }

```

Listing 8: Code for L2 Q2 (Re-entrant Event Firing logic).

Expected Output:

[Check]{High}[Check][Done](Alert)[Done]

Actual Output:

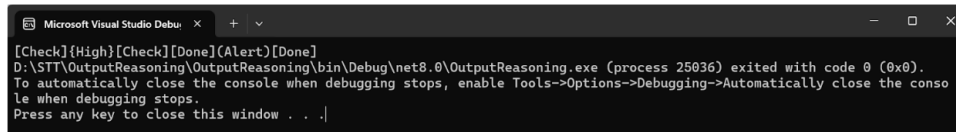


Figure 19: Actual Output for Level 2 Code 2.

Explanation (why): This is another re-entrant call, assuming the `Invoke` is correctly placed inside the `if` block (as is logical).

1. `s.Check(80)` is called.
2. `Check(80)` prints `[Check]`.
3. `if (80 > 50)` is true. `ThresholdReached` fires.
4. Subscriber 1 runs. Prints `{High}`. `e.Info == "High"` is true.
5. `s.Check(30)` is called re-entrantly.
 - `Check(30)` prints `[Check]`.
 - `if (30 > 50)` is false. The event does not fire.
 - `Check(30)` finishes, printing `[Done]`.
6. Control returns to Subscriber 1, which finishes.
7. Subscriber 2 runs, printing `(Alert)`.
8. The original `Check(80)` call resumes and prints `[Done]`.

5 Discussion and Conclusion

5.1 Discussion

This lab was a practical demonstration of a fundamental software design pattern. It's easy to imagine a "simpler" solution where all the logic is placed inside the button's default click event (e.g., `btnChangeColor_Click` would directly set `lblDisplay.ForeColor` and call `MessageBox.Show()`).

However, this "simple" approach creates tightly coupled code that is difficult to maintain and extend. By forcing the use of custom delegates and events, the lab highlights the benefits of a decoupled architecture. The `Form1` class acts as both a publisher (for its buttons) and a subscriber (to its own events). In a larger application (e.g., Model-View-ViewModel or MVVM), the "View" (the Form) would publish events, and a "ViewModel" (a separate class) would subscribe to them to perform business logic. This lab provides the foundational understanding for building such advanced, maintainable applications.

The output reasoning questions were particularly insightful, especially the re-entrant calls in the Level 2 problems. They demonstrate that an event handler's execution can be paused while another event is triggered and fully processed, which can lead to complex state management issues if not handled carefully.

5.2 Limitations and Potential Improvements

- **Naming Convention:** In the provided source code, the handler for `btnChangeText` was named `button1_Click`. While functional, this is a minor inconsistency. It was manually hooked in the constructor, but for future clarity, renaming the method to `btnChangeText_Click` would be better.
- **Standard EventHandler:** For the `TextChangedEvent`, a custom delegate `TextChangedHandler` was created. While this works, C# provides a generic `EventHandler<T>` delegate. We could have used `public event EventHandler<string> TextChangedEvent;`. However, the lab's objective was to practice creating custom delegates, so the current implementation is valid for the assignment.

5.3 Conclusion

This lab helped me successfully build a C# Windows Forms application that uses custom delegates, events, and `EventArgs` to manage UI interactions in a modular, decoupled fashion. The implementation of a multicast event (one event, two subscribers) worked as expected, demonstrating the power of the event-driven model.

The output reasoning exercises further solidified my understanding of how delegate invocation lists are managed, how `ref` parameters behave in this context, and the complex, non-linear execution flow that can result from re-entrant event calls. The lab effectively met all its learning objectives.

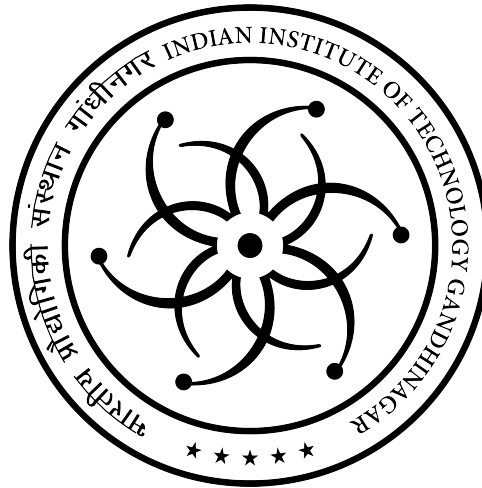
6 References

References

- [1] CS 202 Lab 11 Handout. "Events and Delegates in C# Windows Forms Applications". <https://drive.google.com/file/d/1X-FXS3J02H2rUD041DJb670epbc6h6vc/view?usp=sharing>
- [2] Microsoft Documentation. *Delegates (C# Programming Guide)*. <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/>
- [3] Microsoft Documentation. *Events (C# Programming Guide)*. <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/events/>
- [4] Microsoft Documentation. *How to: Raise and Consume Events*. <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/events/how-to-raise-and-consume-events>
- [5] Microsoft Documentation. *Windows Forms Overview*. <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/>

Lab Assignment - 12

10 November 2025



CS 202

Software Tools and Techniques for CSE

INDIAN INSTITUTE OF TECHNOLOGY GANDHINAGAR

Palaj, Gandhinagar - 382355

Advanced Event Handling and Custom EventArgs in C# Windows Forms Applications

Submitted by

Gaurav Budhwani
22110085

Abstract

This report details the work carried out in Lab 12, focusing on the implementation of advanced event-driven programming concepts within the .NET framework using C#. The primary objective was to design a modular **Order Pipeline** application using Windows Forms that demonstrates the Publisher-Subscriber design pattern, Custom Event Arguments, Event Chaining, and Dynamic Subscription Management.

The lab consisted of two distinct phases. The first phase involved the development of a GUI application where user actions (validation, confirmation, shipping) trigger a chain of custom events. A key feature implemented was the dynamic addition and removal of event handlers for **Express Delivery** notifications based on user selection. The second phase involved a rigorous theoretical analysis of C# delegates, including multicast behavior, exception propagation in delegate chains, and method overloading resolution in inheritance hierarchies.

Contents

1	Introduction, Setup, and Tools	3
1.1	Overview	3
1.2	Objectives	3
1.3	Environment and Tools	3
1.4	One-time Setup Steps	3
2	Methodology and Execution	4
2.1	Part 1: GUI Design and Control Properties	4
2.1.1	Form Layout	4
2.1.2	Input Controls Configuration	5
2.1.3	Action Controls Configuration	6
2.2	Part 2: Code Logic and Implementation	8
2.2.1	Custom Data Transfer: <code>ShipEventArgs</code>	9
2.2.2	Event Definitions and Wiring	9
2.2.3	Event Chaining Logic (The Validation Phase)	9
2.2.4	Dynamic Subscription Logic (The Shipping Phase)	10
3	Results and Analysis	10
3.1	Scenario 1: Invalid Order Handling	10
3.2	Scenario 2: Standard Order Confirmation	11
3.3	Scenario 3: Express Shipping (Dynamic Subscription)	11
3.4	Scenario 4: Standard Shipping (Handler Removal)	12
4	Output Reasoning Exercises	12
4.1	Level 0: Delegate Signatures (Q1) & Multicasting (Q2)	12
4.1.1	Question 1: Delegate Signature Mismatch	12
4.1.2	Question 2: Lambda Multicast Removal	13
4.2	Level 1: Exception Handling (Q1) & Null Parsing (Q2)	13
4.2.1	Question 1: Null String Parsing	13
4.2.2	Question 2: Exceptions in Multicast Chains	14
4.3	Level 2: Inheritance (Q1) & Dynamic Events (Q2)	14
4.3.1	Question 1: Method Overloading and Inheritance	14
4.3.2	Question 2: Dynamic Subscription inside Loop	15
5	Discussion and Conclusion	15
5.1	Discussion	15
5.2	Conclusion	16
6	References	17
7	GitHub Repository	17

1 Introduction, Setup, and Tools

1.1 Overview

Event-driven programming is a paradigm where the flow of the program is determined by events such as user actions, sensor outputs, or message passing. In C# and the .NET ecosystem, events are first-class citizens built upon ‘delegates’. This lab explores advanced scenarios beyond simple button clicks, specifically focusing on passing custom data through events and managing the subscription lifecycle dynamically.

1.2 Objectives

The specific learning outcomes for this lab were:

- **Custom EventArgs:** To define and utilize classes derived from `System.EventArgs` to transport specific data (Product Name, Express Status) from publishers to subscribers.
- **Event Chaining:** To implement a workflow where the completion of one logic block (Validation) automatically triggers the next logic block (Confirmation or Rejection) via events.
- **Dynamic Subscriptions:** To demonstrate the runtime manipulation of invocation lists using `+=` and `-=` operators based on GUI state (CheckBoxes).
- **Output Reasoning:** To analyze and predict the behavior of complex delegate scenarios, including signature mismatches, multicast execution orders, and inheritance shadowing.

1.3 Environment and Tools

The implementation was performed using the following software stack:

- **Operating System:** Windows 11
- **Integrated Development Environment (IDE):** Visual Studio 2022 (Community Edition)
- **Framework:** .NET 8.0 (Windows Forms Application)
- **Programming Language:** C# 10.0 / 12.0

1.4 One-time Setup Steps

To initialize the project:

1. Launched Visual Studio 2022.
2. Selected "Create a new project" → "Windows Forms App".
3. Named the solution `OrderPipeline` and the project `Multi_Stage_Event_Chaining`.
4. Configured the form in the visual designer to accept Order Inputs.

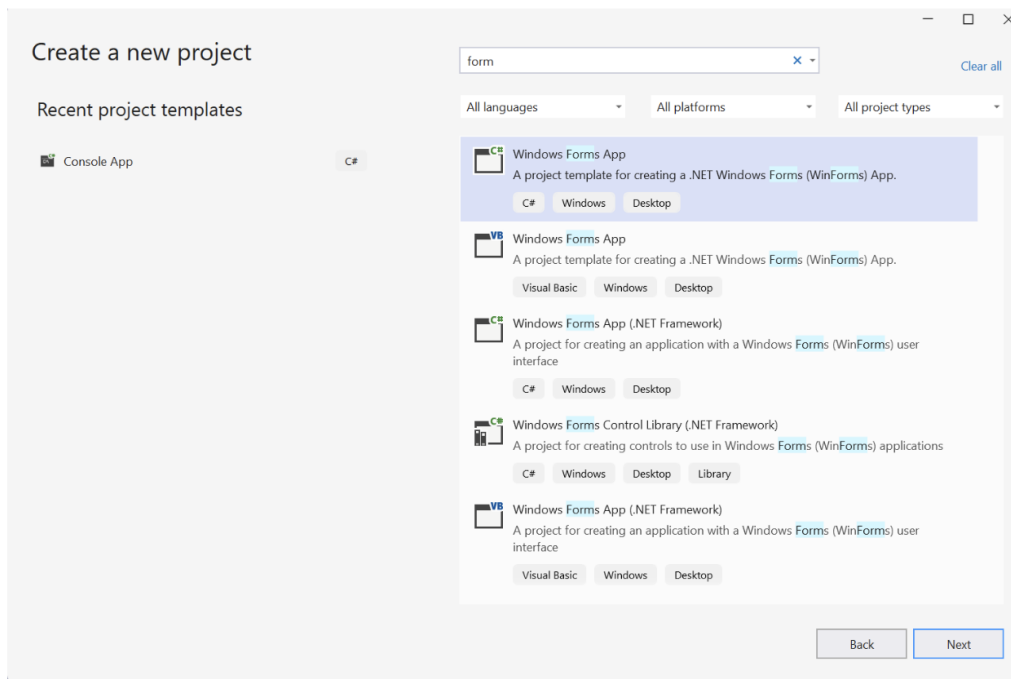


Figure 1: Selecting the "Windows Forms App" template and Saving it as per the required name (Just like Lab 11)

2 Methodology and Execution

This section details the step-by-step construction of the application, highlighting the GUI configuration and the backend logic.

2.1 Part 1: GUI Design and Control Properties

The User Interface was constructed to allow the user to input customer details, select a product type, specify quantity, and toggle delivery options.

2.1.1 Form Layout

The main form, `Form1`, serves as the container for all controls. It was designed to group input fields at the top and action buttons at the bottom.

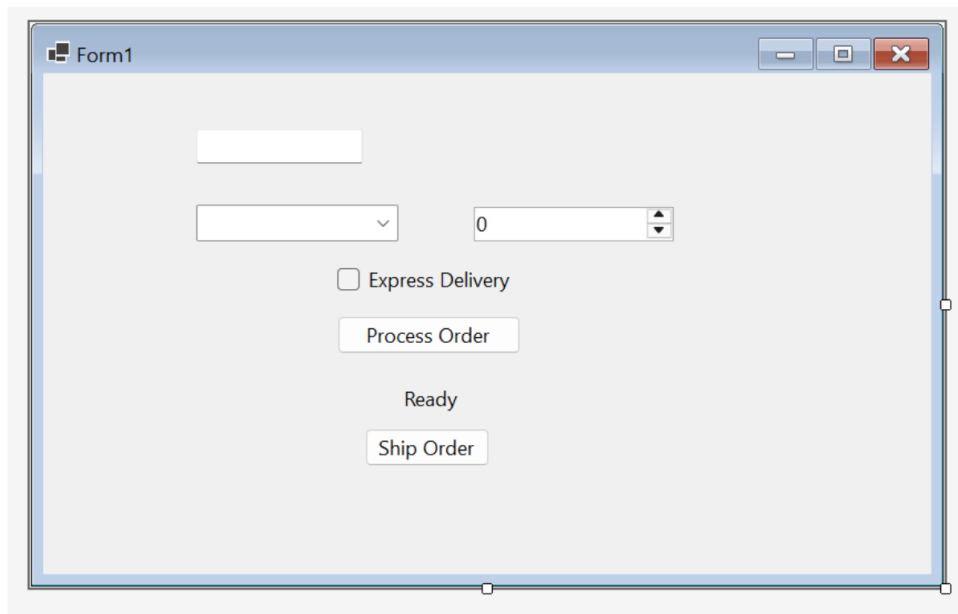


Figure 2: Initial Design of the Order Pipeline Form

2.1.2 Input Controls Configuration

Specific properties were modified to ensure the controls behaved as required by the lab manual.

1. Customer Name (TextBox): Used to accept the customer's string input.

- **Name:** txtCustomerName
- **Location:** (138, 50)
- **Size:** (150, 31)

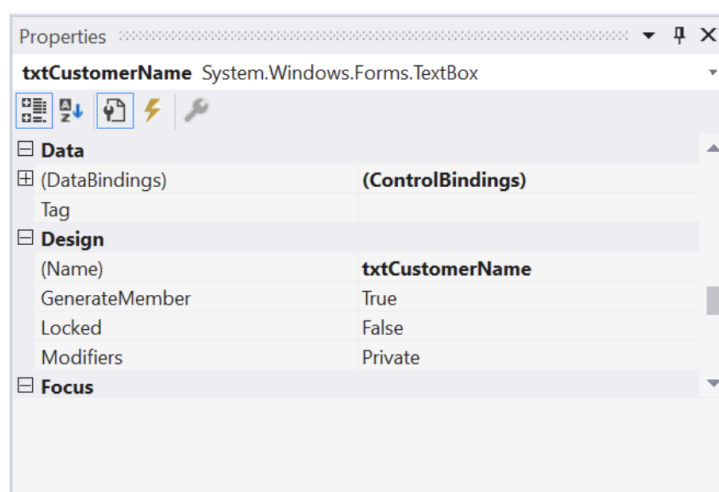


Figure 3: Properties configuration for txtCustomerName

2. Product Selection (ComboBox): Configured with a predefined collection of items.

- **Name:** cmbProduct
- **Items Collection:** "Laptop", "Mouse", "Keyboard"

- **FormattingEnabled:** True

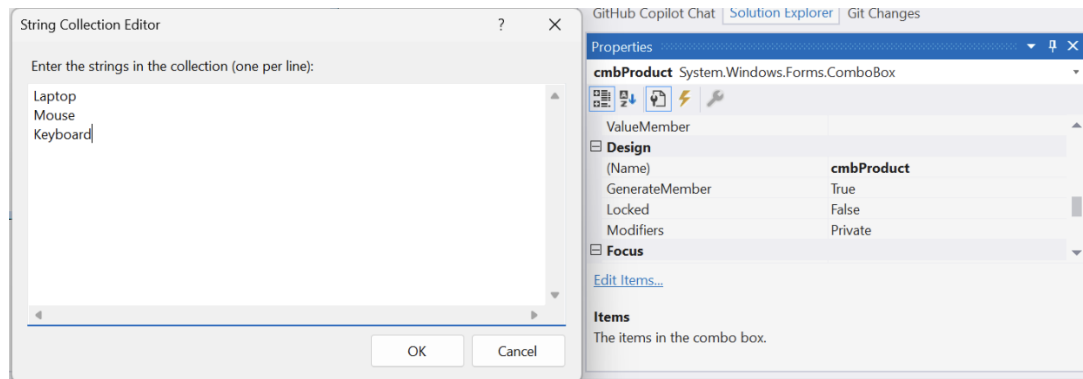


Figure 4: Properties configuration for `cmbProduct` and String Collection Editor

3. **Quantity (NumericUpDown):** Used to enforce integer input for order size.

- **Name:** `numQuantity`
- **Minimum Value:** 0 (Default)
- **TextAlign:** Left

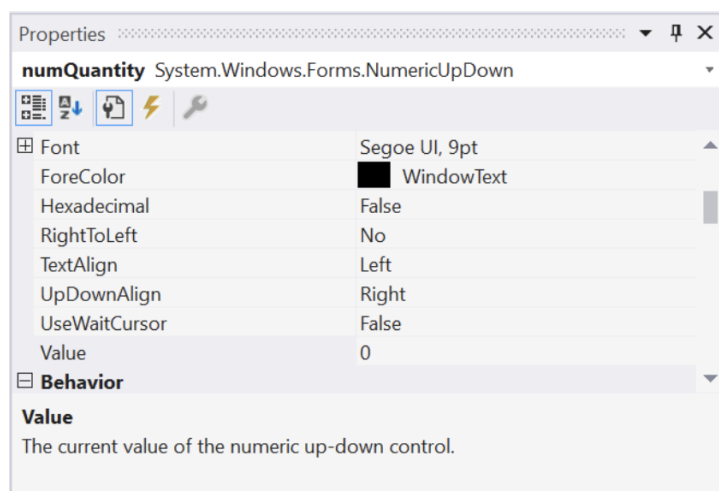


Figure 5: Properties configuration for `numQuantity`

2.1.3 Action Controls Configuration

4. **Process Button:** The trigger for the first stage of the pipeline (Validation).

- **Name:** `btnProcessOrder`
- **Text:** "Process Order"

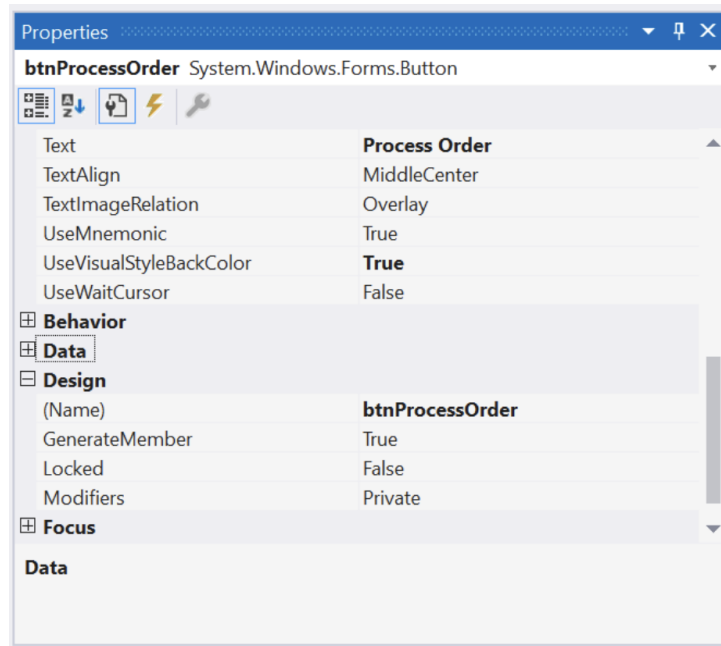


Figure 6: Properties configuration for `btnProcessOrder`

5. Status Label: Used to provide feedback to the user regarding the pipeline state.

- **Name:** `lblStatus`
- **Text:** "Ready"

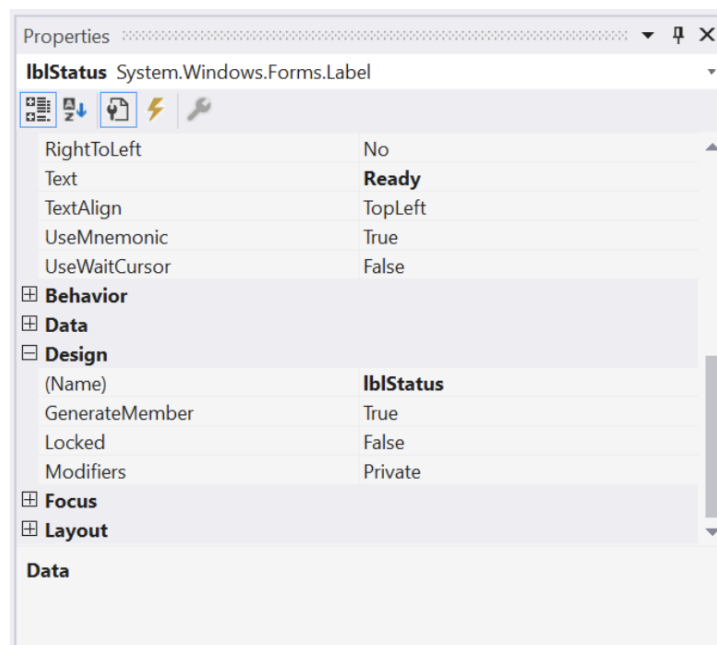


Figure 7: Properties configuration for `lblStatus`

6. Express CheckBox: This control toggles the dynamic subscription logic.

- **Name:** `chkExpress`
- **Text:** "Express Delivery"

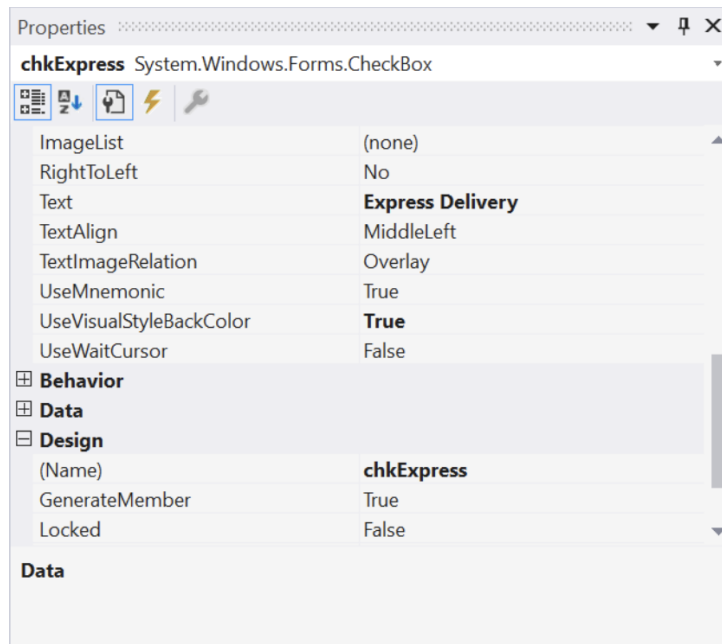


Figure 8: Properties configuration for `chkExpress`

7. Ship Button: The trigger for the final stage of the pipeline (Dispatch).

- **Name:** `btnShipOrder`
- **Text:** "Ship Order"

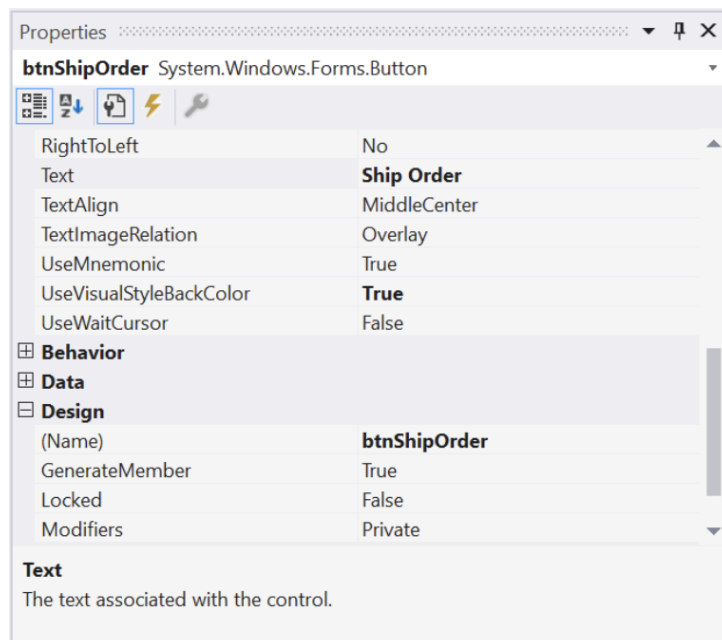


Figure 9: Properties configuration for `btnShipOrder`

2.2 Part 2: Code Logic and Implementation

The logic of the application is divided into three key components: the custom data carrier, the event definitions, and the event handling logic.

2.2.1 Custom Data Transfer: ShipEventArgs

Standard events use `EventArgs`, which carries no data. To meet the requirement of passing the Product name and the Express status, I created a custom class inheriting from `EventArgs`.

```
1 public class ShipEventArgs : EventArgs
2 {
3     public string Product { get; }
4     public bool Express { get; }
5
6     public ShipEventArgs(string p, bool ex)
7     {
8         Product = p;
9         Express = ex;
10    }
11 }
```

Explanation: By making the properties read-only (getter only), we ensure the event data is immutable once the event is raised.

2.2.2 Event Definitions and Wiring

Inside `Form1`, four distinct events were defined to model the pipeline stages:

```
1 public event EventHandler<ShipEventArgs> OrderCreated;
2 public event EventHandler OrderRejected;
3 public event EventHandler OrderConfirmed;
4 public event EventHandler<ShipEventArgs> OrderShipped;
```

Constructor Wiring: To ensure the logic flows correctly upon form load, static subscriptions were added in the constructor. This is preferable to wiring events in the designer as it keeps business logic visible in the code.

```
1 public Form1()
2 {
3     InitializeComponent();
4     // Linking logic blocks to events
5     this.OrderCreated += ValidateOrder;      // Logic: Check input validity
6     this.OrderCreated += DisplayOrderInfo;    // Logic: Show summary popup
7     this.OrderRejected += ShowRejection;      // UI: Update label on failure
8     this.OrderConfirmed += ShowConfirmation;  // UI: Update label on success
9     this.OrderShipped += ShowDispatch;       // UI: Update label on ship
10 }
```

2.2.3 Event Chaining Logic (The Validation Phase)

When the user clicks "Process Order", the button handler does *not* validate the order itself. It merely announces that an order has been created. The subscriber `ValidateOrder` performs the check.

```
1 private void ValidateOrder(object sender, ShipEventArgs e)
2 {
3     // Check if Quantity > 0 and Name is not empty
4     if (numQuantity.Value > 0 && !string.IsNullOrWhiteSpace(txtCustomerName.
5         Text))
6     {
7         // Chain: Validation Success -> Trigger Confirmation
8         isOrderValid = true;
9         OrderConfirmed?.Invoke(this, EventArgs.Empty);
10    }
11    else
12    {
13        // Chain: Validation Failed -> Trigger Rejection
14    }
15 }
```

```

13         isOrderValid = false;
14         OrderRejected?.Invoke(this, EventArgs.Empty);
15     }
16 }

```

Significance: This separation of concerns means the button logic is decoupled from the validation logic. If validation rules change, the button code remains untouched.

2.2.4 Dynamic Subscription Logic (The Shipping Phase)

The "Ship Order" button demonstrates advanced delegate manipulation. The requirement was to notify a courier *only* if the user requested Express Delivery.

```

1 private void btnShipOrder_Click(object sender, EventArgs e)
2 {
3     // Guard Clause: Cannot ship invalid orders
4     if (!isOrderValid) {
5         MessageBox.Show("Cannot ship: No valid order confirmed.");
6         return;
7     }
8
9     // 1. Remove existing handler to prevent duplicates (Safety Check)
10    this.OrderShipped -= NotifyCourier;
11
12    // 2. Conditionally add the handler based on CheckBox state
13    if (chkExpress.Checked)
14    {
15        this.OrderShipped += NotifyCourier;
16    }
17
18    // 3. Raise the event
19    OrderShipped?.Invoke(this,
20        new ShipEventArgs(cmbProduct.SelectedItem?.ToString(), chkExpress.
21        Checked));
22 }

```

Explanation: The line `this.OrderShipped -= NotifyCourier;` is critical. In C#, if you add a handler using `+=` multiple times, it will execute multiple times. By indiscriminately removing it first (which is safe even if it wasn't attached), we ensure that `NotifyCourier` is attached exactly zero times or one time, depending on the checkbox, preventing spam notifications.

3 Results and Analysis

This section documents the successful execution of the "Order Pipeline" application, verifying all functional requirements.

3.1 Scenario 1: Invalid Order Handling

Action: User clicks "Process Order" without entering a name or quantity. **Result:** The Validation logic detects the error. The `OrderRejected` event fires. The Status Label updates to "Order Invalid - Please retry". No confirmation popups appear.

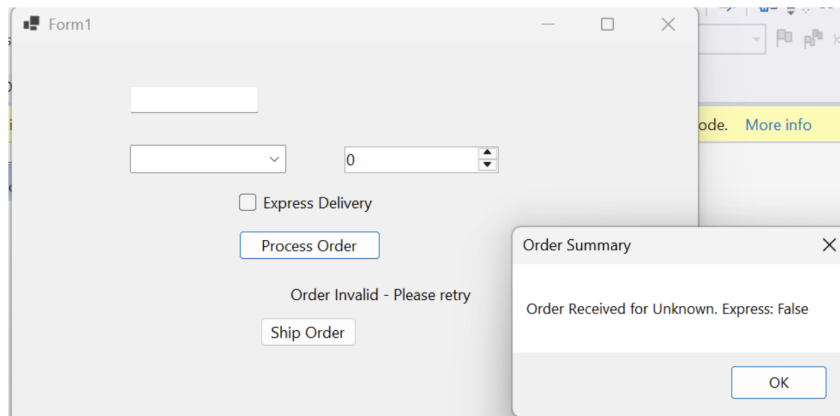


Figure 10: Application state after processing an invalid order

3.2 Scenario 2: Standard Order Confirmation

Action: User enters "Gaurav", selects "Laptop", Quantity 2, and clicks "Process Order". **Result:** Validation passes. `OrderConfirmed` fires. Status Label updates to "Order Processed Successfully for Gaurav".

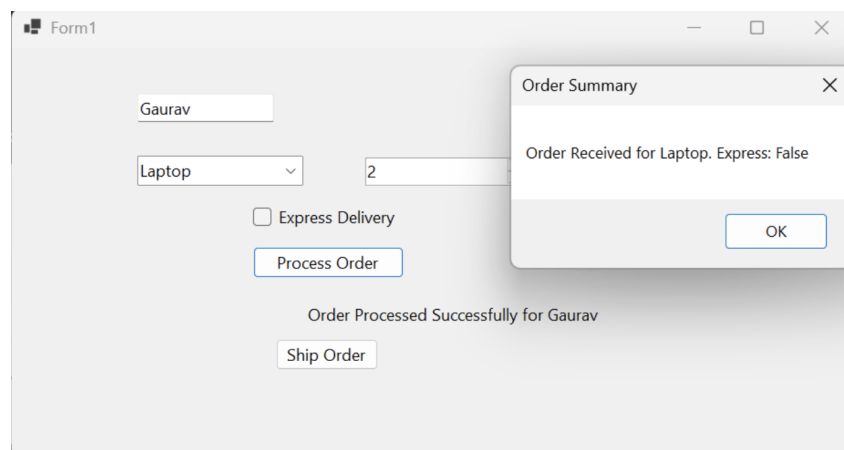


Figure 11: Application state after successfully confirming a standard order

3.3 Scenario 3: Express Shipping (Dynamic Subscription)

Action: User checks "Express Delivery" and clicks "Ship Order". **Result:** The button handler detects the checkbox. It attaches the `NotifyCourier` handler. The event fires. The Courier Notification MessageBox appears ("Express delivery initiated!").

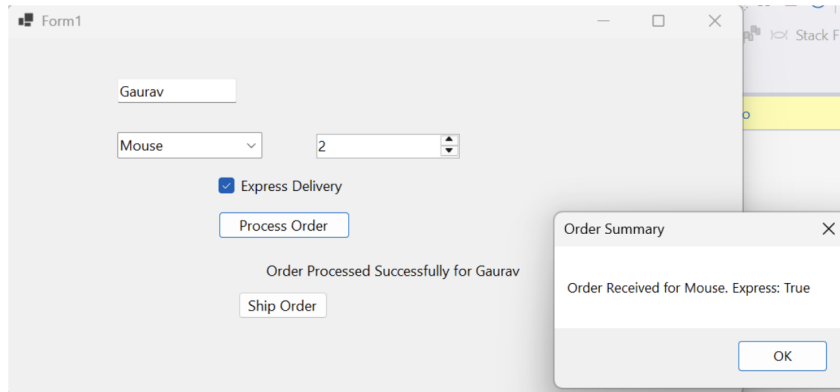


Figure 12: Express Delivery triggers the additional Courier Notification

3.4 Scenario 4: Standard Shipping (Handler Removal)

Action: User unchecks "Express Delivery" and clicks "Ship Order". **Result:** The button handler executes `== NotifyCourier`. The handler is removed. The event fires. **No** Courier Notification MessageBox appears, proving the dynamic removal worked.

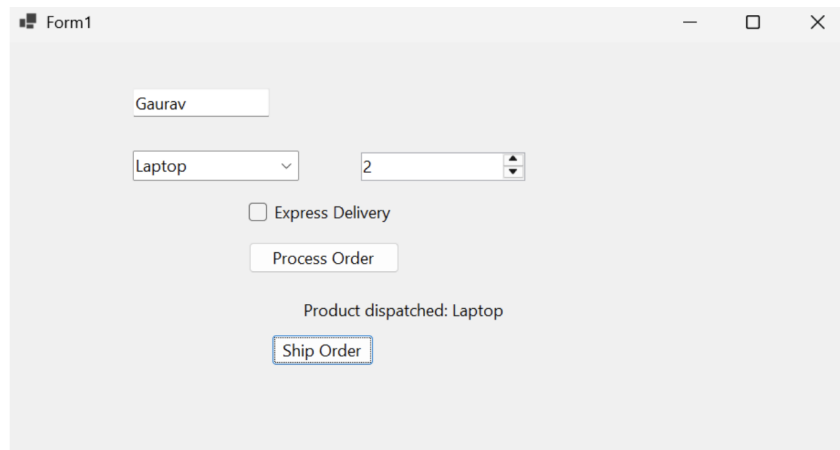


Figure 13: Standard Shipping dispatch message without courier alert

4 Output Reasoning Exercises

This section provides a detailed theoretical analysis of the code snippets provided in the assignment.

4.1 Level 0: Delegate Signatures (Q1) & Multicasting (Q2)

4.1.1 Question 1: Delegate Signature Mismatch

Code Logic: The code defines a delegate `AuthCallback(bool validUser)` which expects one boolean parameter. However, it attempts to assign the method `Login()` (which takes zero parameters) to this delegate.

```
1 public delegate void AuthCallback(bool validUser);
2 public static AuthCallback loginCallback = Login; // Error here
3 public static void Login() { ... }
```

Analysis: In C#, a method is only compatible with a delegate if their signatures match (return type and parameters). Since `AuthCallback` requires 1 argument and `Login` accepts 0, the compiler flags this as a type mismatch.

Observed Output: The code fails to compile.

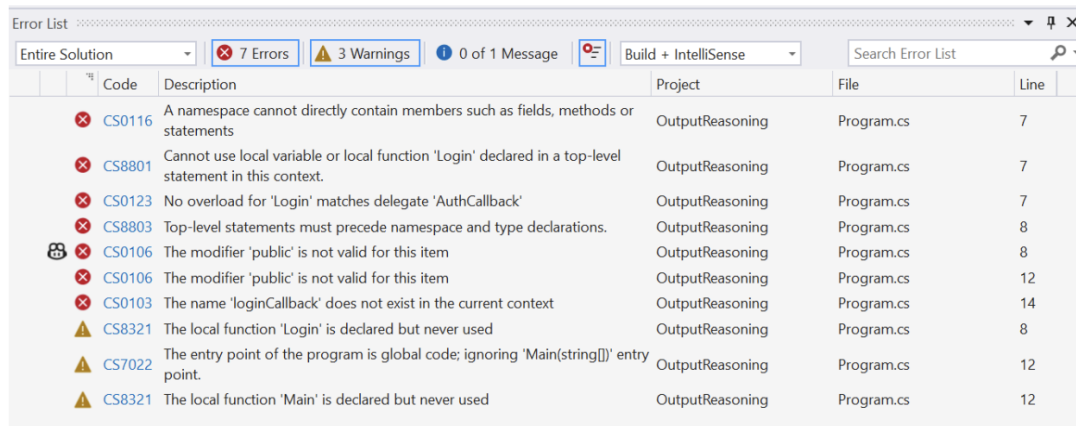


Figure 14: Compilation Error List showing Signature Mismatch

4.1.2 Question 2: Lambda Multicast Removal

Code Logic: A multicast delegate handler is modified using lambda expressions.

```
1 handler += (m) => Console.WriteLine("A:" + m); // Adds Lambda 1
2 handler += (m) => Console.WriteLine("B:" + m); // Adds Lambda 2
3 handler -= (m) => Console.WriteLine("A:" + m); // Attempts to remove Lambda 3
4 handler("world");
```

Analysis: Lambda expressions in C# are anonymous functions. Every time a lambda is defined in code, the compiler generates a new delegate instance (or closure class). The lambda passed to `-=` is a *different object* in memory than the one passed to `+=`, even though the code text is identical. Therefore, the removal fails, and both original lambdas remain.

Observed Output:

```
A: hello (from earlier call)
B: HELLO (from earlier call)
A: world
B: WORLD
```

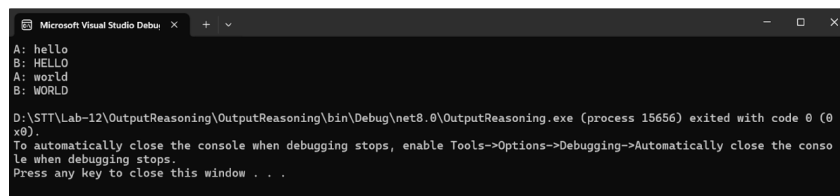


Figure 15: Output proving Lambda identity inequality

4.2 Level 1: Exception Handling (Q1) & Null Parsing (Q2)

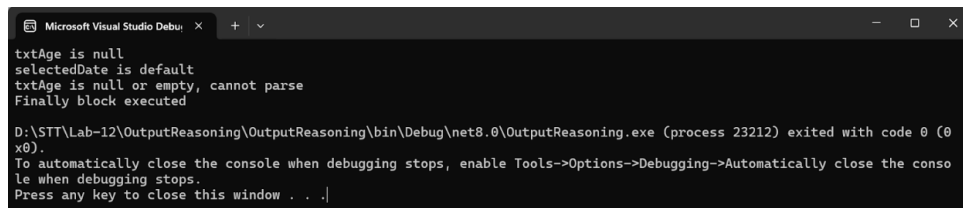
4.2.1 Question 1: Null String Parsing

Code Logic: The code declares `static string txtAge;` without initialization. It then checks `string.IsNullOrEmpty(txtAge)` before attempting to parse.

Analysis:

1. `static string txtAge` defaults to `null`.
2. `string.IsNullOrEmpty(null)` returns `true`.
3. The code enters the `if` block and prints "txtAge is null or empty...".
4. The `else` block containing `int.Parse` is skipped, so no `FormatException` or `ArgumentNullException` is thrown.
5. The `finally` block executes.

Observed Output:



```

Microsoft Visual Studio Debug
txtAge is null
selectedDate is default
txtAge is null or empty, cannot parse
Finally block executed

D:\STT\Lab-12\OutputReasoning\OutputReasoning\bin\Debug\net8.0\OutputReasoning.exe (process 23212) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

```

Figure 16: Output showing safe handling of null strings

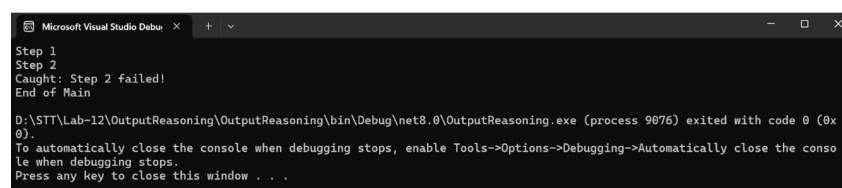
4.2.2 Question 2: Exceptions in Multicast Chains

Code Logic: A multicast delegate `ops` is chained with `Step1`, `Step2`, and `Step3`. `Step2` explicitly throws an exception.

Analysis: When a multicast delegate is invoked, it calls its invocation list sequentially.

- `Step1` executes → Prints "Step 1".
- `Step2` executes → Prints "Step 2" → Throws `InvalidOperationException`.
- The exception bubbles up immediately. The delegate invocation loop terminates.
- `Step3` is **never reached**.
- The `catch` block in `Main` handles the exception.

Observed Output:



```

Microsoft Visual Studio Debug
Step 1
Step 2
Caught: Step 2 failed!
End of Main

D:\STT\Lab-12\OutputReasoning\OutputReasoning\bin\Debug\net8.0\OutputReasoning.exe (process 9076) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

```

Figure 17: Output showing chain interruption upon exception

4.3 Level 2: Inheritance (Q1) & Dynamic Events (Q2)

4.3.1 Question 1: Method Overloading and Inheritance

Code Logic:

```

1 class Base { public void F(int x) ... }
2 class Derived : Base { public void F(double x) ... }
3 // Main:
4 new Base().F(x);    // x is int
5 new Derived().F(x); // x is int

```

Analysis: 1. `new Base().F(x)`: The compiler sees `Base`. It finds `F(int)`. Exact match.
 2. `new Derived().F(x)`: The compiler sees `Derived`. It finds `F(double)`. Note that `F(int)` is in the `Base` class. In C#, unless methods are marked `virtual/override`, a method in a derived class with a compatible signature **hides** the base method by name. Since `int` implicitly casts to `double`, the compiler selects `Derived.F(double)` as the "best match" found on the reference type.

Observed Output:



Figure 18: Output showing Base vs Derived method resolution

4.3.2 Question 2: Dynamic Subscription inside Loop

Code Logic: A loop runs for $i = 1, 2, 3$. Inside the 'StepStarted' event, a `new` subscriber is added to 'StepCompleted' if $i = 2$.

Analysis:

- **i=1:** Start event prints `< S1 >`. Complete event prints `< C1 >`.
- **i=2:** Start event prints `< S2 >`. Code adds generic subscriber '(Dyn)' to Complete event. Complete event fires: Prints original `< C2 >` AND new '(Dyn2)'.
- **i=3:** Start event prints `< S3 >`. Complete event fires. The '(Dyn)' subscriber added in step 2 is **still attached**. Prints `< C3 >` AND '(Dyn3)'.

This illustrates that adding a subscriber inside an event handler persists for future invocations.

Observed Output:

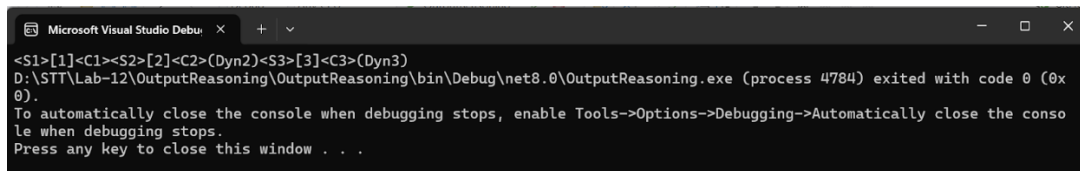


Figure 19: Trace of dynamic event subscription accumulation

5 Discussion and Conclusion

5.1 Discussion

The implementation of the "Order Pipeline" highlighted several critical advantages of the Event-Driven architecture:

1. **Decoupling of Concerns:** The `btnProcessOrder` method acts purely as a trigger. It does not know *how* to validate an order or *how* to display the result. This allows developers to modify validation rules (e.g., adding stock checks) without risking bugs in the UI code.
2. **Strong Typing vs Generic Objects:** The use of `ShipEventArgs` versus generic `EventArgs` significantly improved type safety. Subscribers could access `e.Product` directly without needing error-prone casting.
3. **Runtime Flexibility:** The "Express Delivery" feature demonstrated that software behavior can be modified instantly at runtime by manipulating delegate chains. This is functionally similar to the *Observer Pattern* or *Decorator Pattern* but implemented natively using C# language features.

5.2 Conclusion

This lab helped me build a Windows Forms application that functions correctly, demonstrating event chaining and dynamic subscriber management. The output reasoning exercises provided a necessary theoretical foundation, clarifying how the C# compiler and runtime handle delegates and inheritance. The combination of practical GUI development and theoretical analysis ensures a deep understanding of the .NET Event model.

6 References

References

- [1] Microsoft Documentation. "Handling and Raising Events". [Online]. Available: learn.microsoft.com
- [2] Microsoft Documentation. "Standard .NET Event Patterns". [Online]. Available: learn.microsoft.com
- [3] CS 202 Lab 12 Handout. "Advanced Event Handling and Custom EventArgs in C# Windows Forms Applications". IIT Gandhinagar, 2025. https://drive.google.com/file/d/10MzdiLgt847La0_UitIJ7cnG2Xa8IYo0/view?usp=sharing
- [4] Microsoft Documentation. *How to: Raise and Consume Events*. <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/events/how-to-raise-and-consume-events>
- [5] Microsoft Documentation. *Windows Forms Overview*. <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/>

7 GitHub Repository

The GitHub Repository for this Assignment can be accessed using the following link.
[CLICK HERE](#)